0.1 Introduction

This chapter presents the Capacity Flow Toolkit, which implements the capacity reading of the network model. In this reading the value attached to an edge is the most that can pass along it, and the value attached to a node is the most that can pass through it. The question is how much the network can deliver from its sources to its sinks, and what limits that amount. The toolkit reads the graph object of Chapter 3 directly, uses neither the layered iteration sets nor the Network Decomposition Module of Chapter 4, and is point-valued: every capacity is a single number, and no interval or probability-box propagation is claimed for this reading.

The probabilistic reading of Chapter 5 asks whether supply reaches a node. For a network that carries a resource this is not enough, because a connected network in a degraded state can deliver far less than the demand at its sinks. The capacity reading asks how much is delivered, and the toolkit is a set of modules built on the maximum-flow minimum-cut theory reviewed in Chapter 2. Given a capacitated DAG, it answers six questions:

Q1. What maximum throughput can the network deliver under its current capacities?

Q2. Which cut sets form the binding bottleneck on that throughput, and how many equivalent cuts are there?

Q3. Which component failures or capacity reductions most reduce deliverable flow?

Q4. What capacity change on a given edge maintains, or first reaches, a target service level?

Q5. How much path-disjoint redundancy exists between sources and sinks?

Q6. Which components are single points of failure?

The contributions of this chapter are the following. The flow model is stated with its reductions for several sources and sinks and for node capacities, each proved exact. Three maximum-flow solvers are provided on one graph object, with post-solve checks on every result. The six questions are answered by modules that share one baseline solve. These include an enumeration of every minimum cut through the residual lattice, failure analyses over brute-force and candidate-restricted sets, and a parametric threshold method that uses the piecewise-linear dependence of the maximum flow on a single capacity. Every module is validated against an independent computation on a corpus of twelve networks and on generated instances of up to 10^5 edges, and the toolkit is applied to the IEEE Reliability Test System.

The rest of the chapter is structured as follows. Section 0.2 recalls the results from Chapter 2 on which the toolkit builds. Section 0.3 states the flow model and its two

reductions. Section 0.4 describes the three solvers. Section 0.5 takes the six questions in turn and describes the module that answers each. Section 0.6 gives the cost of every module in terms of solver calls. Section 0.7 describes the validation, and Section 0.8 gives a worked example, the IEEE RTS-24 case study and the scaling results. Section 0.9 discusses limitations and extensions, and Section 0.10 concludes the chapter.

0.2 Background

Chapter 2 reviewed the maximum-flow minimum-cut theorem of Ford and Fulkerson [1, 2] and three results from the same theory that this toolkit uses. A node with its own capacity is handled by splitting it into an entry node and an exit node joined by an edge of that capacity. Any feasible flow decomposes into source-to-sink path flows that sum to the total. The minimum cuts of a network lie between the set of nodes reachable from the source in the residual graph and the set that can reach the sink in the reverse residual graph. Menger's theorem, in its edge and node forms, gives the number of disjoint paths as a unit-capacity maximum flow. The Birnbaum importance of a component [3] is the model for the marginal measures of Section 0.5.3, with the difference that here it is a throughput range without a probability weighting.

Chapter 2 also reviewed the two treatments of uncertain capacities in the literature, the adversarial uncertainty set of robust optimisation [4] and the discrete random capacity of stochastic-flow reliability [5]. Neither is implemented here. The toolkit computes exact answers for a single point-valued capacity assignment, and a best-case and worst-case pair of assignments is two separate solves.

0.3 The Flow Model

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \to \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modeled, let

$$b : V \to \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v. Define the flow variable

$$f : E \to \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u,v))$ for each edge $(u,v) \in E$. For node v, define

$$\delta^+(v) := \{(v,w) \in E\}, \qquad \delta^-(v) := \{(u,v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t, subject to edge capacities c_{uv} for $(u,v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. 1. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The performance measure is the deliverable throughput from s to t, and not only whether an s-t path exists.

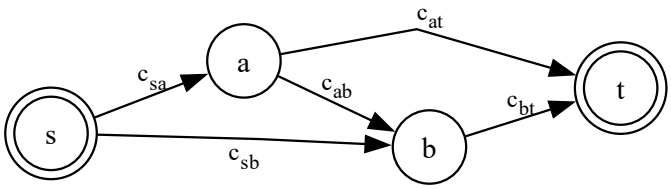


Figure 1: Minimal single-source single-sink capacitated DAG network.

For this base case, let $f : E \to \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t. The optimization model is

$$\max \; F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \qquad \forall (u,v) \in E, \tag{1}$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \qquad \forall v \in V \setminus \{s,t\}, \tag{2}$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \tag{3}$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \tag{4}$$

Taking $F^\star$ to be the optimal (maximum feasible) value of F. Then if the sink node t has a required demand level D_t, the flow module quantifies service adequacy through

the throughput margin

$$\Delta_t := F^\star - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

0.3.1 Several sources and sinks

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure 2a shows the original multi-source multi-sink network.

By introducing a super-source $s^\star$ and a super-sink $t^\star$, as shown in Fig. 2b, the reduced graph is

$$V' = V \cup \{s^\star, t^\star\},$$

$$E' = E \cup \{(s^\star, s) : s \in S\} \cup \{(t, t^\star) : t \in T\}.$$

The connector capacities are set to non-binding values (implemented as Inf in the solver):

$$c_{s^\star s} = +\infty \ \forall s \in S, \qquad c_{tt^\star} = +\infty \ \forall t \in T.$$

(a) Original multi-source multi-sink network.

(b) Equivalent super-source/super-sink reduction.

Figure 2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

Proposition 1 (Exactness of the super-terminal reduction)**.** *The optimal value of the multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals $s^\star$ and $t^\star$.*

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F.

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^\star s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \qquad \forall s \in S,$$

and

$$f'_{tt^\star} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \qquad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, then all capacity constraints from original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from $s^\star$ and the net inflow to $t^\star$ are both equal to F. Therefore f' is a feasible $s^\star$-$t^\star$ flow of value F.

Conversely, let f' be any feasible $s^\star$-$t^\star$ flow on G': then, restricting f' to the original edge set E gives a flow f on G. Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f'. Since feasible flows map in both directions without changing objective value, the two optimal values are equal. □

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max \; F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \qquad \forall (u,v) \in E, \tag{5}$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \qquad \forall v \in V \setminus (S \cup T), \tag{6}$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \tag{7}$$

$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \tag{8}$$

0.3.2 Node capacities by node splitting

The framework accepts values on nodes as well as on edges, so a through-flow limit on a component can be modelled directly. As shown in Figure 3, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+, connected by an internal edge:

$$(v^-, v^+), \qquad c_{v^- v^+} = b_v.$$

The original incoming edges of v are redirected to v^-, and every original outgoing edge is emitted from v^+.

(a) Original network with edge and node capacities.

(b) Equivalent node-split edge-capacitated network.

Figure 3: Node-splitting transformation.

This extends the edge-capacitated flow model with

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \qquad \forall v \in V_c, \tag{9}$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \qquad \forall v \in V_c. \tag{10}$$

Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimization model then becomes

$$\max \; F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \qquad \forall (u,v) \in E, \tag{11}$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \qquad \forall v \in V \setminus \{s,t\}, \tag{12}$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \qquad \forall v \in V_c, \tag{13}$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \qquad \forall v \in V_c, \tag{14}$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \tag{15}$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \tag{16}$$

0.4 Maximum-Flow Solvers

The toolkit provides three solvers for $F^\star$, summarised in Table 1. All three read the graph object of Chapter 3 directly, apply the super-terminal reduction of Section 0.3 when the network has more than one source or sink, and return the flow, the min-cut partition and the flow value. When node capacities are supplied the node-splitting transformation is applied before the solve and the results are mapped back to the original nodes and edges, so that edge flows, sink allocations and cut-side node sets are reported on the network as given.

Solver	Complexity	Best Fit
`:edmonds_karp`	$O(VE^2)$	Small networks
`:dinic`	$O(V^2E)$	Larger layered networks
`:push_relabel`	$O(V^2E)$–$O(V^3)$[a]	Dense, high fan-out networks

[a] Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 1: The three maximum-flow solvers.

0.4.1 Augmenting-path solvers

By the max-flow min-cut theorem of Ford and Fulkerson [1, 2], the maximum flow from s to t equals the smallest total capacity of any set of edges whose removal separates them. The augmenting-path solvers compute both at once.

Definition 1 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \to \mathbb{R}_{\geq 0}$ and a feasible flow f, the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E,\ f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad \textit{(forward residual)}, \\ f_{vu} & (v, u) \in E \quad \textit{(backward residual)}. \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

As Algorithm 1 shows, the augmenting-path solvers repeatedly find a path and push flow along it. Most of the execution time is spent in four lookups:

Algorithm 1 Ford-Fulkerson Min-Cut Max-Flow

1: Initialise residual graph $G_f \leftarrow G$ with zero flow on all edges
2: **while** there exists an augmenting path P from source s to sink t in G_f **do**
3: $\quad \Delta \leftarrow \min_{e \in P} c_f(e)$ ▷ Bottleneck capacity along P
4: $\quad$ **for** each edge $e = (u, v) \in P$ **do**
5: $\quad\quad f(u, v) \leftarrow f(u, v) + \Delta$ ▷ Push flow forward
6: $\quad\quad f(v, u) \leftarrow f(v, u) - \Delta$ ▷ Update reverse edge
7: $\quad$ **end for**
8: $\quad$ Update G_f to reflect new residual capacities
9: **end while**
10: Let A be the set of nodes reachable from s in G_f
11: Min-cut $= \{(u, v) \mid u \in A,\ v \notin A\}$
12: **return** Max-flow value $|f|$, min-cut set A

- Capacity on any edge u-v
- Current flow on any edge u-v
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Fulkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a blocking-flow depth-first search. The level graph restricts traversal to edges $u \to v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor `ptr[u]` to track each node's position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework's default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

0.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

Algorithm 2 Push-Relabel Min-Cut Max-Flow

height$[s^\star] \leftarrow |V|$; height$[v] \leftarrow 0 \; \forall \, v \neq s^\star$
Push maximum admissible flow from $s^\star$ to all neighbours; initialise excess$[v]$
while $\exists$ active node u with excess$[u] > 0$ **do**
    if $\exists$ neighbour v with $r(u, v) > 0$ and height$[u] =$ height$[v] + 1$ **then**
        $\delta \leftarrow \min(\text{excess}[u], r(u, v))$
        Push δ units from u to v; update excess$[u]$, excess$[v]$, $r(u, v)$, $r(v, u)$
    else
        height$[u] \leftarrow 1 + \min_{v:\, r(u,v)>0}$ height$[v]$ ▷ Relabel
    end if
end while
Traverse final flow state from $s^\star$ to identify min-cut partition
return Max-flow value $|f|$, min-cut set

0.5 The Questions the Toolkit Answers

The six questions of Section 0.1 are answered by modules that share the solvers of Section 0.4. Figure 4 shows how the modules are called from one baseline solve. This section takes the questions in turn; the throughput question is the baseline solve itself and the margin Δ_t of Section 0.3.

0.5.1 The binding cut and how many equivalent cuts there are

The flow module solvers all compute the max flow $F^\star$ by constructing a representative valid min-cut $S^\star$ in the given network G. This output by itself neither indicates nor guarantees that $S^\star$ is the only possible min-cut set in G.

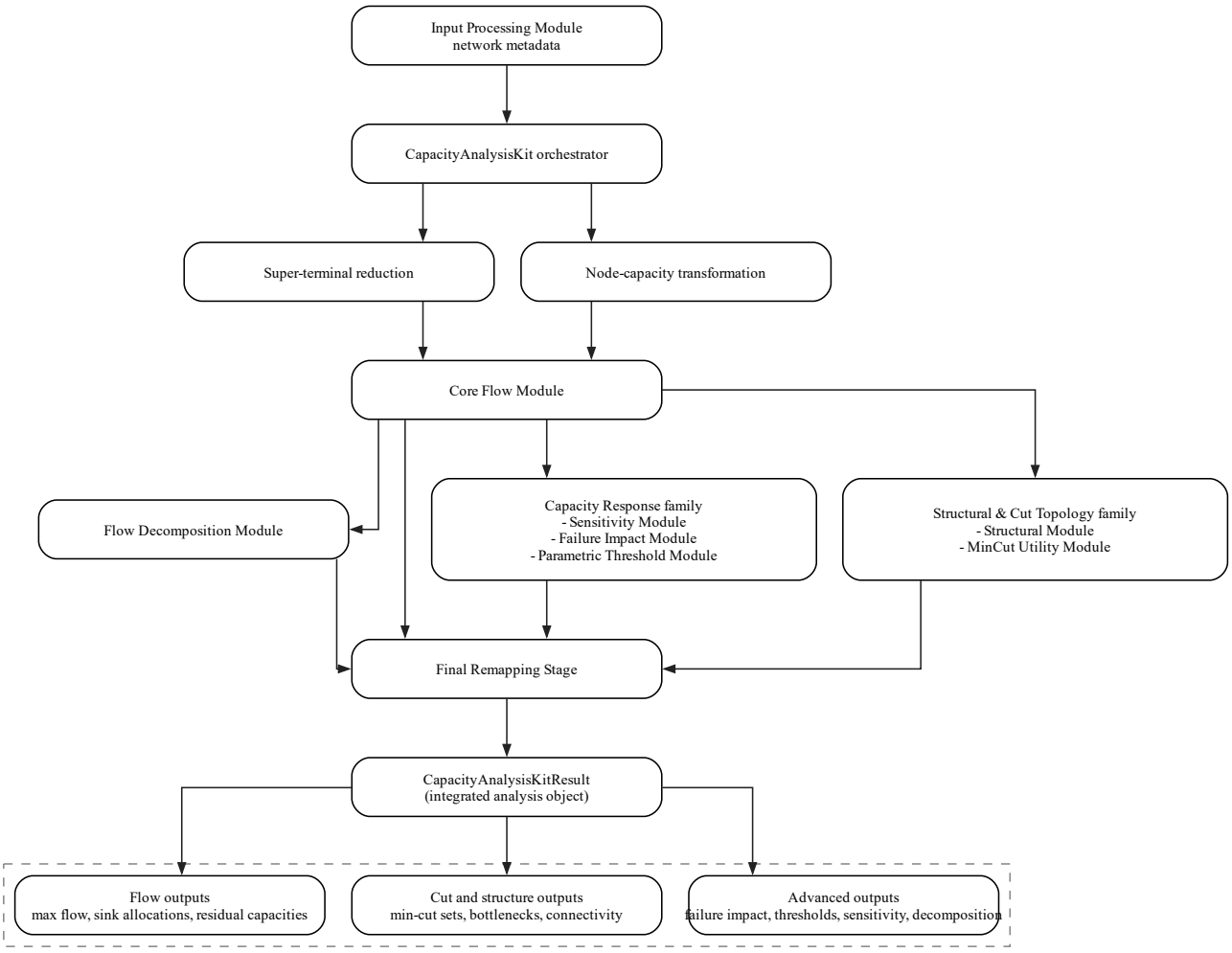


Figure 4: Runtime workflow for the flow-capacity toolkit with available modules

The distinction matters for engineering decisions. Infrastructure systems are designed with redundancy, and in such networks several structurally distinct partitions of V commonly have the same cut capacity $F^\star$.

While the baseline max flow value directly addresses the first analysis objective, the utility module exploits the already-generated min-cut and residual graph to build the extended partition $S^{\star\star}$ and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from $t^\star$ in G_f.

Definition 2 (Extended Source Partition)**.** *The extended source-side region is*

$$S^{\star\star} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity.

Consequently $S^\star \subseteq S^{\star\star}$.

Definition 3 (Free Zone)**.** *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{\star\star} \setminus S^\star.$$

Nodes in $\mathcal{F}$ *can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of* G *under* $f^\star$ *takes the form* $S = S^\star \cup R$ *for some* $R \subseteq \mathcal{F}$.

The free zone size $|\mathcal{F}|$ itself determines the degeneracy of the bottleneck structure. Specifically, when $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and $S^\star$ is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition $S^{\star\star}$ and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. It then follows that two distinct edge sets can be derived directly from from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u,v) \in E : f^\star_{uv} = c_{uv},\ u \in S^{\star\star},\ v \notin S^\star\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u,v) \in E : f^\star_{uv} = c_{uv},\ u \in S^\star,\ v \in T^{\star\star}\},$$

where $T^{\star\star} = V \setminus S^{\star\star}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^\star \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq$ `cut_limit` all minimum cuts are returned and `is_complete` = `true`; otherwise, the first `cut_limit` cuts are returned with `is_complete` = `false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

0.5.2 Redundancy and single points of failure

The structural module identifies components whose importance follows from their position in G alone, whatever the capacities.

Understanding the route-level structure of G is a prerequisite for any path-level fragility or flow-distribution analysis. The module exposes a base path enumerator function

that performs a depth-first search on the topologically sorted iteration set derived from the `InputProcessingModule`. Since G is a DAG, exact enumeration is always finite; however, highly redundant networks can admit exponentially many routes, so a `path_limit` parameter bounds the number of enumerated paths and returns a sample and not the complete set $\mathcal{P}$.

Flow Decomposition. Two networks can share identical topology and the same $F^\star$ while concentrating flow through entirely different routes. During design decisions where multiple configurations are evaluated and per-route visibility is needed, this decomposition exposes how $f^\star$ is distributed across $\mathcal{P}$.

Given an optimal flow $f^\star$ and a set of flow-agnostic paths $\mathcal{P}$, for each path $P = (v_0, v_1, \ldots, v_k) \in \mathcal{P}$, the bottleneck flow contribution is the minimum flow on any edge along P:

$$\phi(P) = \min_i f^\star_{v_i v_{i+1}},$$

Thus, the edge meeting this minimum must be the path's local bottleneck. Paths are sorted in descending order of $\phi(P)$ as a ranked decomposition of $f^\star$ into per-route contributions. Where the min-cut module's ranking informs global capacity upgrade decisions, this ranking informs targeted interventions at the route level. In a network supplying two distinct demand nodes via separate routes, the min-cut module points to the tightest cut edge across the whole network, whereas the decomposition identifies independently which component is constraining delivery to each demand node.

Structural Single Point of Failure Nodes.

Definition 4 (Structural Single Point of Failure)**.** *A node $v \in V \setminus (S \cup T)$ is a SPOF node if $F^\star(G \setminus \{v\}) = 0$, where removing v also removes all incident edges. A SPOF node is one through which every source-to-sink path must pass; its removal alone disconnects the network regardless of any capacity assignment.*

The candidate set is restricted to nodes lying on at least one s-t path,

$$\mathcal{C} = \big(\mathrm{fwd}(S) \cap \mathrm{bwd}(T)\big) \setminus (S \cup T),$$

where $\mathrm{fwd}(S)$ and $\mathrm{bwd}(T)$ are the sets of nodes forward-reachable from S and backward-reachable from T in G respectively. Nodes outside $\mathcal{C}$ lie on no s-t path and cannot be SPOF nodes. For each $v \in \mathcal{C}$, a BFS is run on $G[V \setminus \{v\}]$; if no sink in T is reachable from any source in S, v is recorded as a SPOF node. The function requires one BFS per candidate and runs in $O(|\mathcal{C}| \cdot (|V| + |E|))$ time.

Noted that in a node-capacitated network, a node appearing in the min-cut is not necessarily a SPOF node in this structural sense. A capacity-constrained node enters

the cut-space by way of its split edges $(v_{\text{in}}, v_{\text{out}})$ with capacity b_v and is not necessarily a structural choke-point in G. In these networks, a node can simultaneously be a flow-limiting bottleneck and a SPOF node.

0.5.3 Which edges limit throughput, and by how much

At optimality, the flow $f^\star$ partitions the edge set into two classes: edges carrying strictly less than their capacity, $f^\star_{uv} < c_{uv}$, and edges that carry exactly their capacity,

$$f^\star_{uv} = c_{uv}.$$

The latter are referred to as *saturated edges* in this work. By the max-flow min-cut theorem, every minimum cut set is made up only of saturated edges; therefore, any augmenting path from S to T must cross at least one saturated edge. Since the binding constraint lies elsewhere, *unsaturated edges* carry slack and are transparent to perturbation ; increasing or decreasing their capacity by a small enough amount has little to no effect on $F^\star$. The Flow Sensitivity module focuses only on saturated edges, since they collectively form the capacity-critical boundary of the network. This module exposes the following three functions.

Critical Edge Ranking. A direct measure of an edge's failure impact is how much the max flow drops when that edge is disconnected from the network, referred to as edge criticality in this module. The function re-runs the flow solver on a modified network for each candidate. For each saturated edge (u, v), the critical impact is:

$$\Delta^\star_{uv} = F^\star(G,\, c) - F^\star\Big(G,\, c_0^{(uv)}\Big)\,,$$

where $c_0^{(uv)}$ denotes the capacity function with c_{uv} set to zero and all other capacities unchanged. The function returns the edges in descending order of $\Delta^\star_{uv}$.

Marginal Capacity Values. Similar to the critical edge ranker, the function is scoped to saturated edges. For each saturated edge (u, v), it quantifies the incremental gain in $F^\star$ per unit of additional capacity,

$$\mu_{uv} = \frac{F^\star(G,\, c_\delta^{(uv)}) - F^\star(G,\, c)}{\delta},$$

where $c_\delta^{(uv)}$ increases c_{uv} by δ with all other capacities unchanged.

In highly redundant networks, bypass routes often exist that can absorb rerouted flow and upgrade priorities are not always clear. In these systems, an edge can rank high for failure impact but have a lower marginal value. The critical edge function identifies which

edges are most damaging to lose while the marginal capacity function complements this by identifying which ones are most worth upgrading.

Marginal Range. Both $\Delta^\star$ and μ are snapshot evaluations relative to the current capacity assignment c. Depending on the network structure, an edge with a low μ under the current inputs may become highly influential under different capacity values. This Marginal Range function evaluates the total range of influence a single edge (u, v) can exert on $F^\star$:

$$\mathcal{B}_{uv} = F^\star\big(G,\, c_\infty^{(uv)}\big) - F^\star\Big(G,\, c_0^{(uv)}\Big)\,,$$

where $c_\infty^{(uv)}$ sets $c_{uv} = \infty$ and $c_0^{(uv)}$ sets $c_{uv} = 0$, and all other capacities fixed. This is function is analogous to the classical Birnbaum importance measure $I_B(i) = \partial h(\mathbf{p})/\partial p_i$ (see [3]). However, the analogy is conceptual since $\mathcal{B}_{uv}$ is a deterministic throughput range, not a probability-weighted sensitivity.

0.5.4 The most damaging failures and degradations

The failure impact module models outright component failures and degraded operating states.

Single and Combined Edge Failures. For a single failed edge (u, v), the throughput impact is:

$$\Delta_{uv} = F^\star(G,\, c) - F^\star\Big(G,\, c_0^{(uv)}\Big)\,,$$

where $c_0^{(uv)}$ sets $c_{uv} = 0$ with all other capacities unchanged and candidate edges are restricted to E_{some}.

Real failure events are not always isolated to a single component. A storm or substation fault in a power network can take down multiple edges simultaneously. Since the interaction between affected edges may open or close re-routing options, the combined impact of k simultaneous failures is not in general the sum of their individual Δ_{uv} impacts.

The k-edge failure function generalises the single failure case to any set of k edges removed simultaneously. For a combination $\mathcal{K} \subseteq E_{\text{some}}$ with $|\mathcal{K}| = k$, the combined impact is:

$$\Delta_{\mathcal{K}} = F^\star(G,\, c) - F^\star\Big(G,\, c_0^{(\mathcal{K})}\Big)\,,$$

where $c_0^{(\mathcal{K})}$ sets $c_{uv} = 0$ for all $(u, v) \in \mathcal{K}$. All $\binom{|E_{\text{some}}|}{k}$ combinations require one re-solve each. In practice, because $\binom{|E_{\text{some}}|}{k}$ grows rapidly with k, this function is most tractable for $k = 2$ or $k = 3$ depending on the size and density of the network. The function returns a complete ranking of k-edge failure scenarios by throughput impact.

Capacity Degradation Scenarios. Infrastructure components do not always fail completely. Partial blockage or degraded components are more likely to present as a reduction in c_{uv} without complete loss. Additionally, a domain expert working across multiple network states may want to compare the effect of varying input capacities and operating conditions. This function is intended to be used to efficiently compute the full $F^\star$ picture across all scenarios without needing to re-run the input processing or baseline solve steps for each. The degradation function accepts either an explicit override dictionary specifying new capacities for individual edges, or a uniform scale factor $\alpha \in [0, 1]$ applied to all finite capacities,

$$c'_{uv} = \alpha \cdot c_{uv}, \qquad \forall (u, v) \in E.$$

0.5.5 Degradation and upgrade thresholds

The previous modules treat edge capacities as fixed and analyse the network's response to perturbations around them. The threshold module inverts this perspective. Given a required demand level $D_t \leq F^\star$ to be met, it determines what capacity each edge must have for the network to meet that target delivery level. It is therefore a planning tool.

This module exploits a specific structural property of $F^\star$: it is piecewise linear in any single edge capacity c_e with all others fixed. Logically, each linear segment maps to a fixed min-cut partition $S^\star$, over which the threshold satisfying $F^\star(c_e) = D_t$ can be recovered analytically. Therefore, breakpoints must occur exactly where the min-cut partition shifts, i.e. where a change in c_e causes a different set of edges to become the binding bottleneck.

Degradation Threshold. For a target edge (u, v) and a required flow level D_t, the degradation threshold $c^\dagger_{uv}$ is the minimum capacity the edge must retain for $F^\star$ to remain at least D_t:

$$c^\dagger_{uv} = \inf\big\{c \in [0, c_{uv}] : F^\star(G,\, c_c^{(uv)}) \geq D_t\big\},$$

where $c_c^{(uv)}$ denotes the capacity function with c_{uv} replaced by c. The degradation margin $c_{uv} - c^\dagger_{uv}$ is the slack available before the service level is breached. The module ranks edges by ascending degradation margin, so the most vulnerable edges appear first. A small margin can be interpreted as the edge is close to becoming a binding constraint on D_t, while a large margin means the edge can absorb substantial capacity loss without service impact.

The threshold is located by a partition-change recursion. Starting from the known endpoints $c_e = 0$ and $c_e = c_{uv}$, the algorithm evaluates $F^\star$ at the midpoint and compares the resulting min-cut partition to those at the endpoints. If both endpoints share the same partition, the segment is linear and the threshold is solved analytically. If the partitions differ, the interval contains a breakpoint and the recursion descends into the

sub-interval containing D_t. This continues until the threshold is isolated to a partition-constant segment, where it is computed exactly. The result is exactly capacity value at which $F^\star$ crosses D_t.

Two degenerate cases are handled explicitly. If $F^\star(c_e = 0) \geq D_t$, the edge can degrade to zero without breaching the target. If $F^\star(c_e = c_{uv}) < D_t$, the target is unachievable even at the current capacity, meaning the bottleneck lies entirely elsewhere and this edge cannot help.

Upgrade Threshold. If $F^\star < D_t$ under the current capacity assignment, the upgrade threshold is the minimum capacity (u, v) must have for $F^\star$ to first reach D_t. Since the required capacity may exceed c_{uv}, the upper search bound is not known in advance. The algorithm uses a doubling search to bracket the threshold interval, repeatedly doubling the trial capacity until $F^\star \geq D_t$, then applies the same partition-change recursion to locate it exactly.

An edge case arises when the edge is not on any minimum cut: expanding its capacity cannot raise $F^\star$ regardless of how large c_{uv} becomes. The doubling search detects this when $F^\star$ stops increasing before reaching D_t and returns `upgrade_ineffective` = `true`, indicating that reaching D_t requires addressing a different bottleneck.

When no explicit target is supplied, the module defaults to $D_t = 0.9 \cdot F^\star$.

0.6 Cost

The total computational cost of any derived analysis is the product of the per-solve cost of the chosen max-flow algorithm and the number of solver calls required. Repeated-solve tasks scale with solve count, which is the dominant cost driver in practice. Per-solve costs are summarised in Table 1.

Module	**Function**	**Operation**	**Complexity**
Base Solver			
Base	Edmonds-Karp	BFS augmenting path	$O(VE^2)$
Base	Dinic	Level graph + blocking	$O(V^2E)$
Base	Push-Relabel	Preflow relabeling	$O(V^3)$
Node-Cap	Split transformation	Graph transform	$O(V + E)$
Node-Cap	Solve (split)	Solver on $2V, 2E$	$O(T_{\text{solver}})$
Min-Cut Utility			
Min-Cut	`cut_edges`	Extract partition	$O(E)$
Min-Cut	`edges_some`	BFS + filter	$O(E + T_s)$
Min-Cut	`edges_every`	Reachability	$O(V + E)$

Module	Function	Operation	Complexity
Min-Cut	`enumerate`	Lattice ($2^{\lvert\mathcal{F}\rvert}$)	$O(2^{\lvert\mathcal{F}\rvert} \cdot E)$[1]
Structural			
Struct	`spof_nodes`	BFS per candidate	$O(\lvert\mathcal{C}\rvert(V+E))$
Struct	`enumerate_paths`	DFS on DAG	$O(\lvert\mathcal{P}\rvert)$[2]
Struct	`path_flow`	Per-path scan	$O(\lvert\mathcal{P}\rvert\ell)$
Struct	`redundancy`	Menger (unit cap)	$O(\lvert E_c\rvert T_s)$
Flow Sensitivity			
Sens	`critical_edges`	Zero-cap solves	$O(\lvert E_{\text{sat}}\rvert T_s)$
Sens	`marginal_capacity`	$+\delta$ solves	$O(\lvert E_{\text{sat}}\rvert T_s)$
Sens	`marginal_range`	Range evals	$O(\lvert E_{\text{some}}\rvert T_s)$
Failure Impact			
Fail	`single_edge`	Per-edge solves	$O(\lvert E_{\text{some}}\rvert T_s)$
Fail	`k_edge`	Per-combination	$O(\binom{\lvert E_s\rvert}{k} T_s)$[3]
Fail	`degradation`	Per-scenario	$O(\lvert S\rvert T_s)$
Parametric Threshold			
Param	`degrade`	Partition recursion	$O(\lg(c/\varepsilon)T_s)$
Param	`upgrade`	Doubling + recurse	$O(\lg^2(c/\varepsilon)T_s)$
Param	`all_degrade`	Per edge	$O(\lvert E_c\rvert \lg(c/\varepsilon)T_s)$

Table 2: Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.

0.7 Validation

Every output of the toolkit was checked against a computation that does not use its solvers. The checks were run on a set of twelve networks, the ten DAGs of the Chapter 7 validation set with the three published drone designs in place of the larger drone network, plus the PSPLIB instance of Chapter 7, each with integer capacities drawn uniformly from $[1, 20]$ per edge from a recorded seed, and four of them with node capacities drawn in the same way. The networks, seeds and files are listed in Appendix B.

The maximum-flow value was computed by all three solvers and by the `GraphsFlows.jl` library [6] as an external oracle. On every network the three solvers agree with each other and with the oracle exactly, and every returned flow passes the three post-solve checks: capacity feasibility on every edge, conservation at every transit node, and equality of the flow value and the reported cut capacity.

The structural outputs were checked by brute force. Every enumerated minimum cut was checked to have capacity equal to the maximum flow and to disconnect the sources from the sinks, and on networks with at most twenty candidate edges every edge subset that disconnects was enumerated and the reported cuts were confirmed to be exactly the minimal ones of minimum capacity. Every reported saturated edge was checked to carry flow equal to its capacity. Single points of failure and the single-edge failure ranking were checked by removing each node and each edge in turn and recomputing the maximum flow with the oracle, exhaustively on the small networks and on a sample of between fifteen and forty removals on the large ones. Each degradation threshold was checked by evaluating the oracle just below and just above it, and by locating the threshold independently by bisection. The edge-disjoint path count was checked against a unit-capacity oracle flow, which is Menger's theorem. The node-capacitated flow was checked against the oracle applied to the split graph, and the several-source and several-sink construction is exercised by every network in the set. All checks passed on all twelve networks.

The last of these checks found an error. On the IEEE network of Section 0.8.2, where ten of twenty-four buses carry a node capacity and the rest do not, the node-splitting transformation assigned the entry and exit nodes identifiers $2v$ and $2v+1$, which collide with the identifiers of unsplit nodes when only some nodes are split and the identifiers are small and consecutive. None of the twelve corpus networks had exposed it, because in each of them every node or no node carried a capacity. The transformation now assigns identifiers above the largest identifier in the network, and the corrected code gives identical results on every network that had passed before. The change is described in Chapter 8.

0.8 Case Studies

Timings in this section are wall-clock measurements on a single core of the second call in a warm process, so that just-in-time compilation is excluded. The convention is stated in Appendix B.

0.8.1 A worked example on a synthetic network

The modules are first shown together on a synthetic network built so that each of them has something to report. It is a worked example; the case study follows in Section 0.8.2. The network is a multi-source, multi-sink capacitated DAG with 43 nodes, 107 directed edges. The benchmark simulates a distribution system that propagates resources across geographically dispersed locations. As shown in Figure 6, it is structured as an 8-layer hierarchy:

- **Origin nodes** (blue): five source nodes, over-provisioned so the supply zone does not form the binding bottleneck.
- **Hub nodes** (green): eight intermediate relay nodes with high fan-out capacities.
- **Processing nodes** (red, node-capacitated) and **bypass nodes** (orange, node-capacitated): the processing corridor carries the bulk of flow; the bypass is a lightweight alternative with tighter node constraints.
- **Trunk and secondary nodes** (purple, brown): distribute flow through a reconverging structure toward the gateway layer.
- **Dispatch gateways** (light blue, node-capacitated): two gateways assigned equal edge capacity of 17 by design.
- **Distribution combiner** (pink): aggregates all gateway flow before fan-out to the sinks.
- **Sink nodes** (grey): seven demand nodes, each with incoming edge capacity 6.

Based on this layered structure, edge and node capacity assignments are chosen so that binding bottlenecks emerge at specific layers under the different analysis methods. Edge capacities in Layers 1–5 are scaled by a factor of 2 relative to the gateway layer. In this way, the gateway layer governs $F^\star$ under the edge-only solution. Similarly, to compute a measurable reduction in $F^\star$, the processing node capacities are set to bind to the node-capacitated solve.

To explore competing design alternatives under controlled conditions, two configurations of the same system are presented to evaluate the effect of feeder reinforcement (Figures 6a and 6b). Both configurations keep the same topology and differ in their feeder capacities. The comparison tests whether the bottleneck structure changes when the throughput does not.

Config A proposes a symmetry-preserving design with balanced feeder-edge capacities and a limited direct bypass to sink 4. The capacity is set below the sink's 6-unit limit so that the bypass delivers partial service. Config B's asymmetric gateway structure is designed to produce non-trivial parametric upgrade thresholds and non-zero marginal-range values. This version has larger feeder capacities from secondary nodes into both gateways.

Table 3 summarises the key structural differences between the two. The expectation is that two designs may yield the same max-flow value while differing in saturation pattern and in the set of minimum cuts, because their feeder capacities differ.

Two configurations compared

The base flow solver run returns $F^\star = 37.0$, with `mincut_capacity` $= 37.0$ for both networks, consistent with the expected maximum feasible flow for the benchmarked system. However, when optimising for a demand target or comparing two network design approaches, $F^\star$ alone is not always enough to inform decisions. As shown in Table 3, the two networks still differ in structure: Configuration A has 41 saturated edges versus 35 in Configuration B, and 4 valid minimum cuts versus 1. This is also reflected in free-zone size (2 in A, 0 in B). The same throughput under structurally different constraints motivates the use of the extended analysis modules in the toolkit.

Table 3: Configuration comparison summary.

Metric	Config A	Config B
F^*	37.0	37.0
`saturated_edges`	41	35
`free_zone_size`	2	0
`min_cuts`	4	1
`SPOF_nodes`	0	0
`upgrade_ineffective` (gateway edges)	`true`	`false`

From a design perspective, this reflects a trade-off between structural flexibility and tighter bottleneck localisation. In Configuration A, the free-zone nodes are $\{134, 135\}$, producing a wider cut lattice where E_{every} and E_{some} differ. The gateway edges $134 \to 136$ and $135 \to 136$ appear in E_{some} but not in E_{every}, so neither is a universal failure mode on its own. Cut 4 has only 3 crossing edges while Cut 1 has 13, both at capacity 37 (Table 8). In Configuration B, feeder-capacity reinforcement pins the cut to a unique partition: free-zone size collapses to 0 and the gateway edges move into E_{every}. The choice between configurations depends on the optimisation target: Configuration A offers more structural redundancy and cut diversity, while Configuration B offers tighter bottleneck localisation and actionable upgrade thresholds.

Bypass edge and single points of failure. The direct bypass edge $132 \to 140$ improves `sink_4` service with capacity 3, below the sink-side limit of 6. The same edge raises redundancy scores for all secondary-to-gateway connections from 2 to 3, improving redundancy across the gateway-feeding layer, not only the directly connected nodes.

Without this edge, node 136 is the only SPOF node: every source-to-sink path passes through it and $F^\star(G \setminus \{136\}) = 0$. Adding $132 \to 140$ gives $F^\star(G \setminus \{136\}) = 3 > 0$, removing node 136 from the SPOF set, and both configurations report `spof_nodes` $= \emptyset$. However, only `sink_4` has an alternative path, so all other flow still passes through the combiner. Running the capacity degradation module with node 136 removed confirms

this: $F^\star$ drops to 3, exposing the network's continued vulnerability to combiner failure despite the absence of a formal SPOF classification.

Node-capacitated analysis. When node capacities are enabled, both configurations still return the same result: a 7-unit loss down to `node_cap_max_flow` = 30.0. The same six Layer 3 nodes $\{114, 115, 116, 117, 118, 119\}$ saturate in both cases. This matters because it tells the system designer where to act: once these node limits are active, changing gateway-edge capacities does not change the outcome.

The flow loss is not shared evenly across demand nodes (Figure 9). `sink_1` (137) falls from 4.0 to 0.0, so it loses service completely. `sink_2` (138) falls from 6.0 to 3.0, while sinks 139–143 stay at their previous levels. Reporting total flow alone misses the main service issue.

To test a direct fix, a single-node upgrade sweep was run on nodes 114–119 (Table 4). Any one of these nodes can restore `sink_1` to its edge-only service level with $\Delta b_v = 7.0$, and this also returns total flow to $F^\star = 37.0$ in both configurations. The increment is the same for all six nodes, but the final capacity targets are not: nodes 114/115 go to $b_v = 12.0$, nodes 116/117 go to $b_v = 14.0$, and nodes 118/119 go to $b_v = 10.0$. In practice, that means the choice should be made by delivery constraints such as cost, installation effort, and access, not by throughput effect.

Table 4: Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore `sink_1` service under the node-capacitated solve.

Node	Current b_v	Required Δb_v	New b_v
114 (`processing_A`)	5.0	7.0	12.0
115 (`processing_B`)	5.0	7.0	12.0
116 (`processing_C`)	7.0	7.0	14.0
117 (`processing_D`)	7.0	7.0	14.0
118 (`bypass_E`)	3.0	7.0	10.0
119 (`bypass_F`)	3.0	7.0	10.0

Flow decomposition results. Flow decomposition shows that both configurations achieve the same total throughput ($F^\star = 37.0$) with very similar route-level allocation. Configuration A decomposes into 27 components and Configuration B into 25, so the difference is small and does not change the main delivery pattern. In both cases, the largest single component carries 3.0 units on a path bottlenecked at $(114 \to 120)$, and most remaining components are paths carrying 2.0 or 1.0 units distributed across the processing and bypass corridors.

This supports the earlier structural findings. The A/B edge-capacity changes alter which secondary-to-gateway edges appear as local path bottlenecks, but they do not

create a different high-level flow-allocation picture. The dominant route family still passes through the same Layer 3 processing/bypass region and then through the combiner to sinks, with the direct bypass to sink 4 contributing only partial service. At decomposition level, the two designs differ in local edge pressure more than in overall route usage.

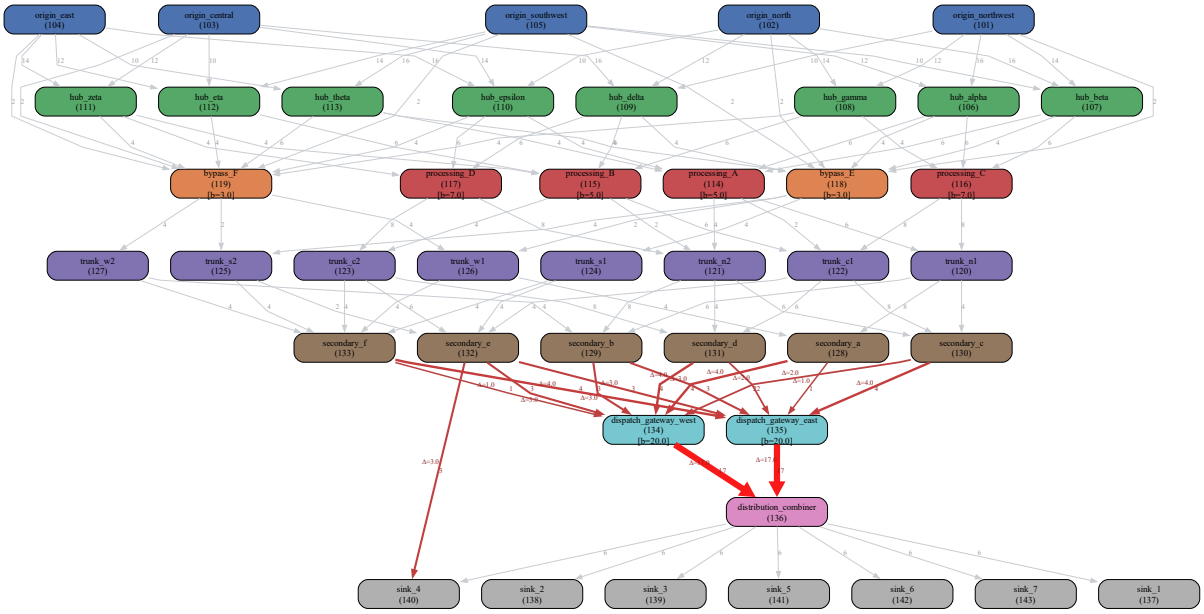
Sensitivity analysis. Sensitivity ranking shows a clear tiered pattern in Config A (Table 5). The two gateway-to-combiner edges (134 → 136) and (135 → 136) rank first and second with $\Delta^\star = 17.0$ each, reducing flow from $F^\star = 37.0$ to 20.0 when removed (about 46%). The next tier contains four feeder edges with $\Delta^\star = 4.0$ and perturbed flow 33.0: (128 → 134), (130 → 135), (131 → 134), and (133 → 135). A third tier of five edges has $\Delta^\star = 3.0$ and perturbed flow 34.0, including the direct bypass edge (132 → 140); this places the bypass in the same sensitivity tier as several secondary-to-gateway feeders. The remaining non-zero edges have $\Delta^\star \in \{1.0, 2.0\}$ with perturbed flow 35.0 or 36.0. In total, 15 edges have non-zero single-edge impact in Configuration A.

Table 5: Full ranked single-edge sensitivity results for Configuration A (all non-zero $\Delta F^\star$ edges).

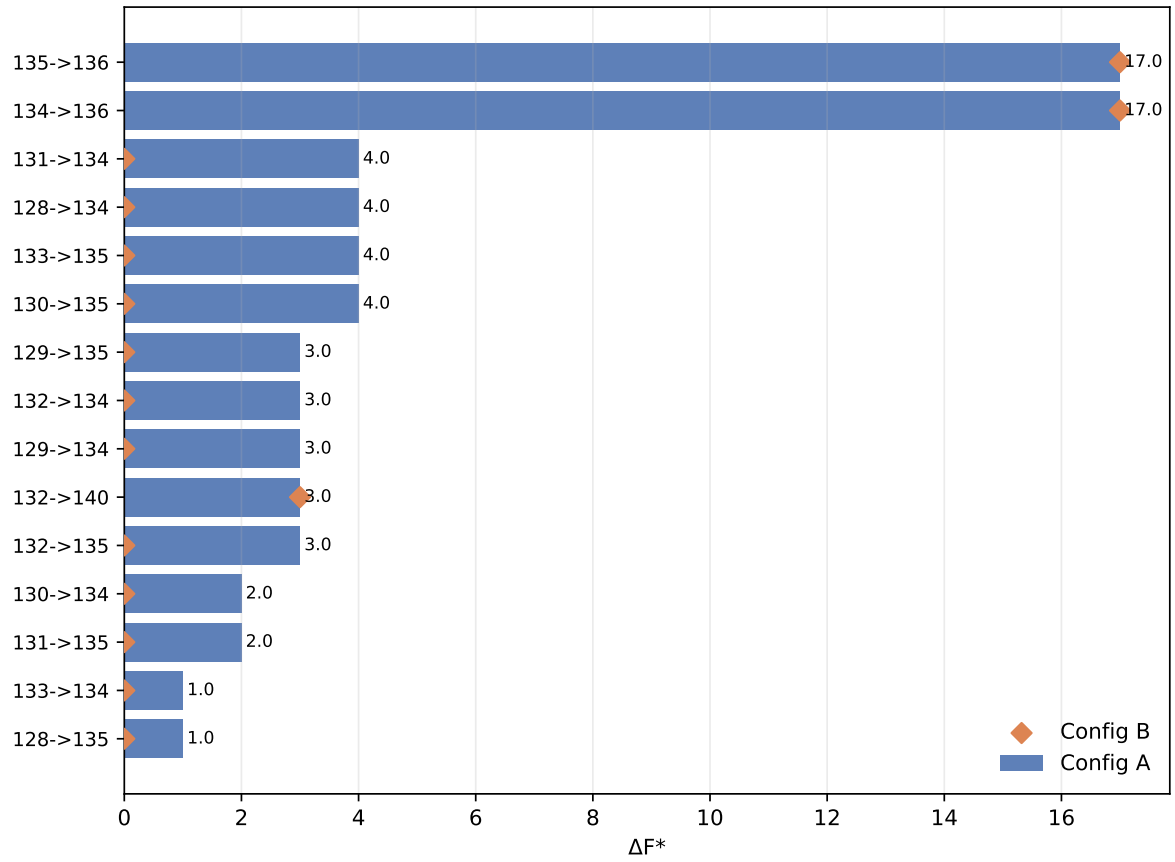
Rank	Label	$\Delta F^\star$	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_a->dispatch_gateway_west	4.0	33.0
4	secondary_c->dispatch_gateway_east	4.0	33.0
5	secondary_d->dispatch_gateway_west	4.0	33.0
6	secondary_f->dispatch_gateway_east	4.0	33.0
7	secondary_b->dispatch_gateway_west	3.0	34.0
8	secondary_b->dispatch_gateway_east	3.0	34.0
9	secondary_e->dispatch_gateway_west	3.0	34.0
10	secondary_e->dispatch_gateway_east	3.0	34.0
11	secondary_e->sink_4	3.0	34.0
12	secondary_c->dispatch_gateway_west	2.0	35.0
13	secondary_d->dispatch_gateway_east	2.0	35.0
14	secondary_a->dispatch_gateway_east	1.0	36.0
15	secondary_f->dispatch_gateway_west	1.0	36.0

Configuration B is much more concentrated: only three edges have non-zero impact, (134 → 136), (135 → 136), and (132 → 140), while all other evaluated edges give $\Delta^\star = 0$ (Table 6). This means the feeder-capacity reinforcement in B absorbs any single feeder-edge loss through rerouting, so feeder removals no longer reduce $F^\star$ individually. The non-zero set therefore collapses from 15 edges in A to 3 in B.

Failure impact. The two-edge failure ranking confirms that the dominant pair is simultaneous loss of (134 → 136) and (135 → 136), with drop 34.0 and perturbed flow 3.0.



(a) Configuration A critical-edge overlay. Edge emphasis (colour/width) is proportional to non-zero $\Delta F^{\star}$.



(b) Ranked non-zero single-edge sensitivity for Configurations A and B.

Figure 5: Single-edge sensitivity in Config A and cross-configuration rank comparison (A vs B).

Table 6: Full ranked single-edge sensitivity results for Configuration B (all non-zero $\Delta F^\star$ edges).

Rank	Label	$\Delta F^\star$	`perturbed_flow`
1	`dispatch_gateway_west->distribution_combiner`	17.0	20.0
2	`dispatch_gateway_east->distribution_combiner`	17.0	20.0
3	`secondary_e->sink_4`	3.0	34.0

This is much larger than the next group (drop 21.0), and makes sense given the single-edge pattern shown in Figure 5. In Configuration B, the same gateway-pair dominance remains, while the non-zero failure set is more concentrated because feeder reinforcement absorbs most single-edge feeder losses through rerouting.

Degradation trajectory. The degradation trajectories are identical for Configurations A and B at all tested scale factors. The solved values are: $\alpha = 1.0 \to 37.0$, $\alpha = 0.9 \to 33.3$, $\alpha = 0.8 \to 29.6$, $\alpha = 0.7 \to 25.9$, $\alpha = 0.6 \to 22.2$, and $\alpha = 0.5 \to 18.5$. These points follow the linear relation

$$F^\star(\alpha) = 37\alpha$$

over the tested interval.

The A/B curves that coincide point-for-point (Figure 8) indicate that the feeder-capacity differences that separate A and B under single-edge sensitivity do not produce a different global response under a uniformly scaled network-wide degradation.

Parametric thresholds. Parametric thresholds separate edge criticality from edge upgradeability. In Configuration A, the gateway edges $(134 \to 136)$ and $(135 \to 136)$ are upgrade-ineffective (`required_capacity` $= \infty$, `ineffective`=`true`) even though they are high-impact in sensitivity ranking. In Configuration B, the same gateway edges become upgrade-effective with `required_capacity`=18.0 and `required_increase`=1.0. Degradation thresholds remain at 17.0 with zero margin in both configurations, so any capacity loss below current levels immediately reduces $F^\star$.

This difference is explained by upstream supply. In Configuration A, supply into each gateway is already capped at the current gateway-edge capacity, so increasing gateway-edge capacity alone cannot raise throughput. In Configuration B, feeder reinforcement creates upstream headroom, so gateway-edge upgrades can increase $F^\star$.

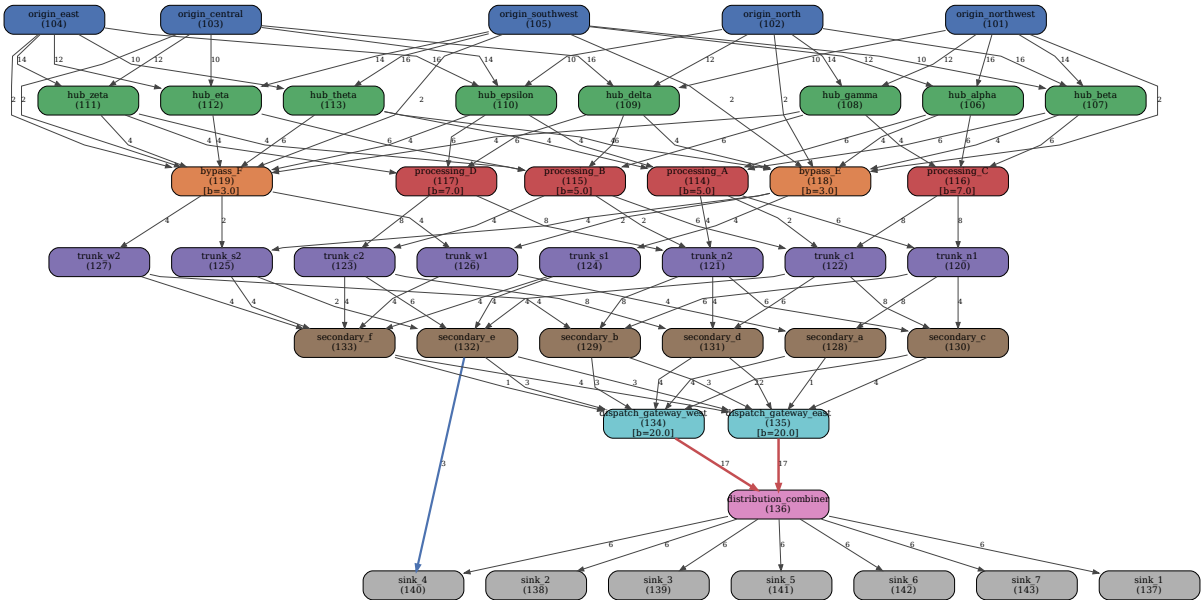
For the bypass edge $(132 \to 140)$, $\mathcal{B}_{uv}$ increases from 10.0 (A) to 16.0 (B), and the edge is upgrade-effective in both configurations with `required_increase`=1.0 (`required_capacity`=4.0). The main A/B contrast therefore remains at the gateway edges, not at bypass upgrade feasibility.

Table 7: Parametric threshold and marginal range comparison for key edges.

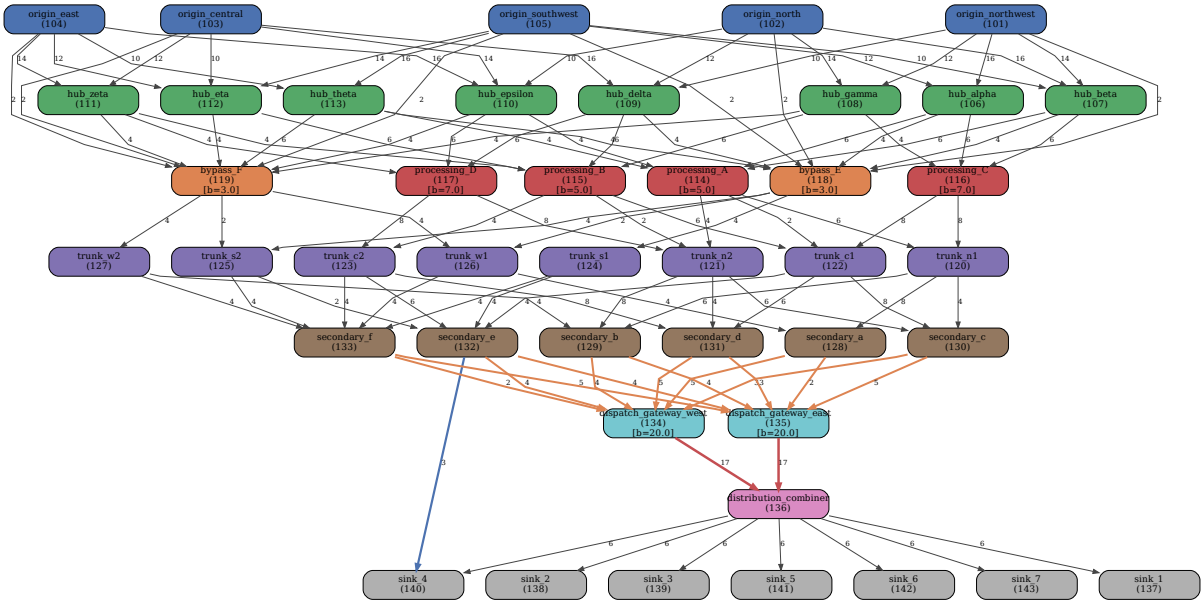
Edge	Config A		Config B	
	Upgrade	$\mathcal{B}_{uv}$	Upgrade	$\mathcal{B}_{uv}$
$134 \to 136$	ineffective	17.0	$+1.0$	23.0
$135 \to 136$	ineffective	17.0	$+1.0$	23.0
$132 \to 140$	$+1.0$	10.0	$+1.0$	16.0

Table 8: Min-cut enumeration for Config A.

Cut	Crossing edges
1	$(128 \to 134)$, $(128 \to 135)$, $(129 \to 134)$, $(129 \to 135)$, $(130 \to 134)$, $(130 \to 135)$, $(131 \to 134)$, $(131 \to 135)$, $(132 \to 134)$, $(132 \to 135)$, $(132 \to 140)$, $(133 \to 134)$, $(133 \to 135)$
2	$(128 \to 135)$, $(129 \to 135)$, $(130 \to 135)$, $(131 \to 135)$, $(132 \to 135)$, $(132 \to 140)$, $(133 \to 135)$, $(134 \to 136)$
3	$(128 \to 134)$, $(129 \to 134)$, $(130 \to 134)$, $(131 \to 134)$, $(132 \to 134)$, $(132 \to 140)$, $(133 \to 134)$, $(135 \to 136)$
4	$(132 \to 140)$, $(134 \to 136)$, $(135 \to 136)$



(a) Configuration A: symmetric gateway design. Blue edge 132 → 140 is the direct bypass to `sink_4`.



(b) Configuration B: orange edges show capacity-boosted secondary-to-gateway feeders

Figure 6: Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.

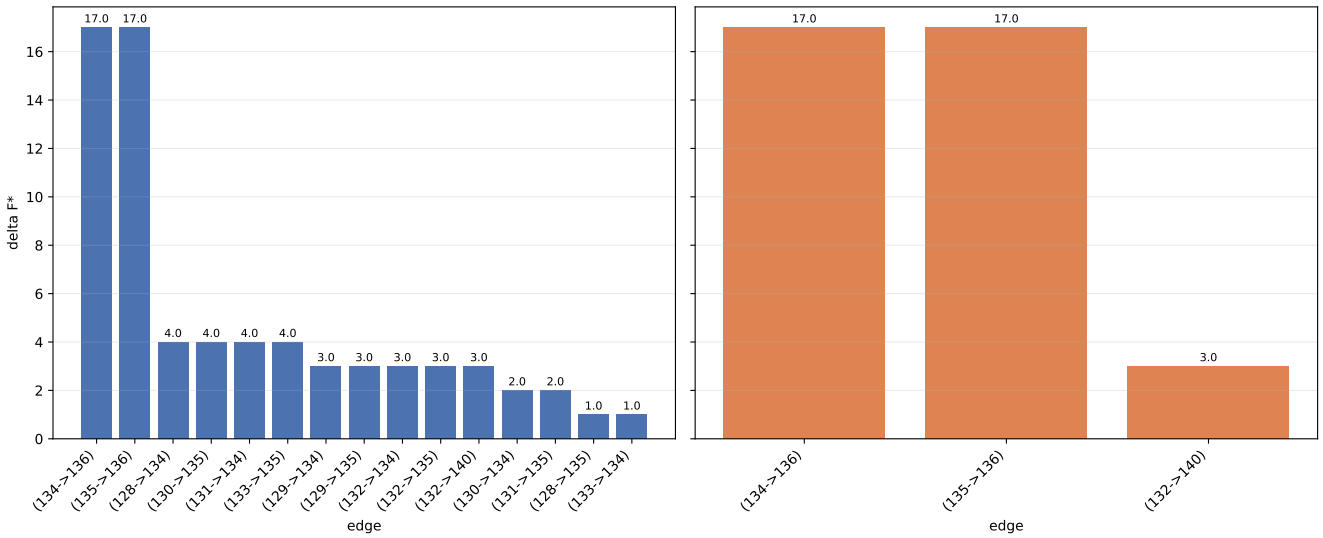


Figure 7: Sensitivity ranking for non-zero $\Delta F^\star$ values in Configurations A and B.

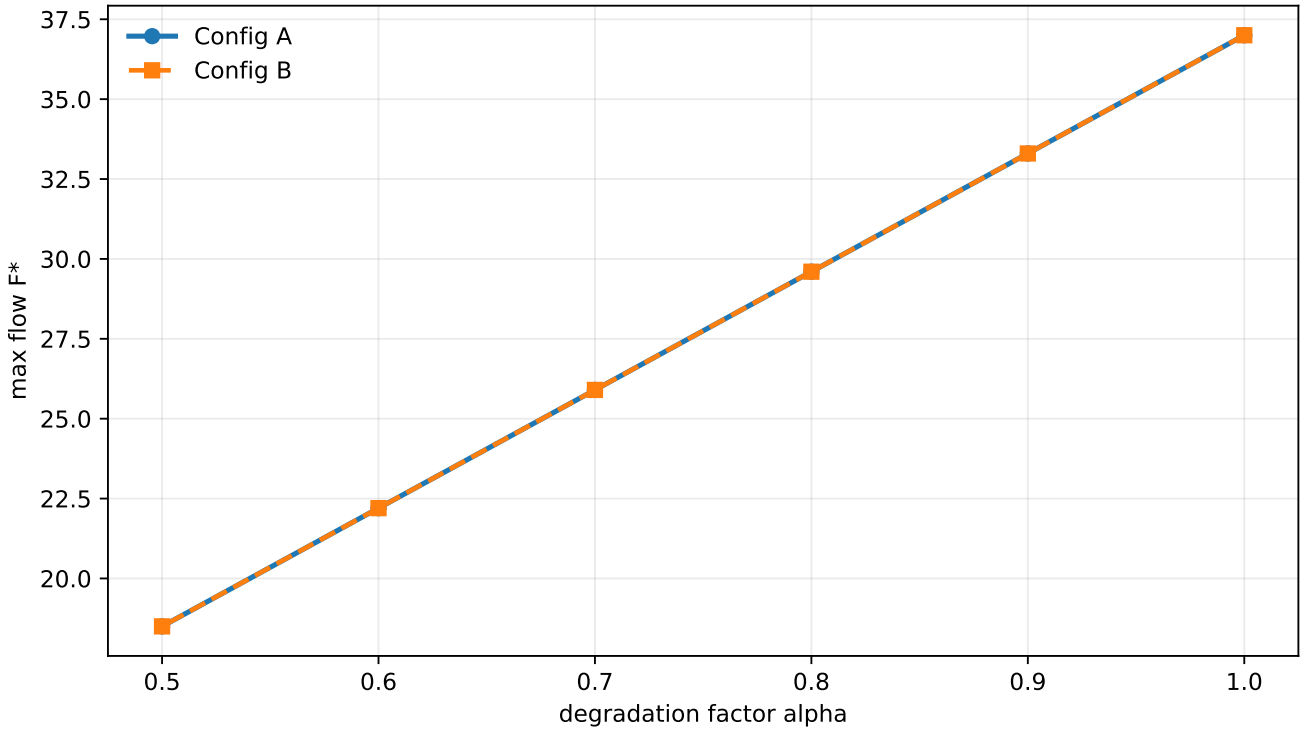


Figure 8: Degradation trajectory $F^\star(\alpha)$ for Configurations A and B under uniform capacity scaling.

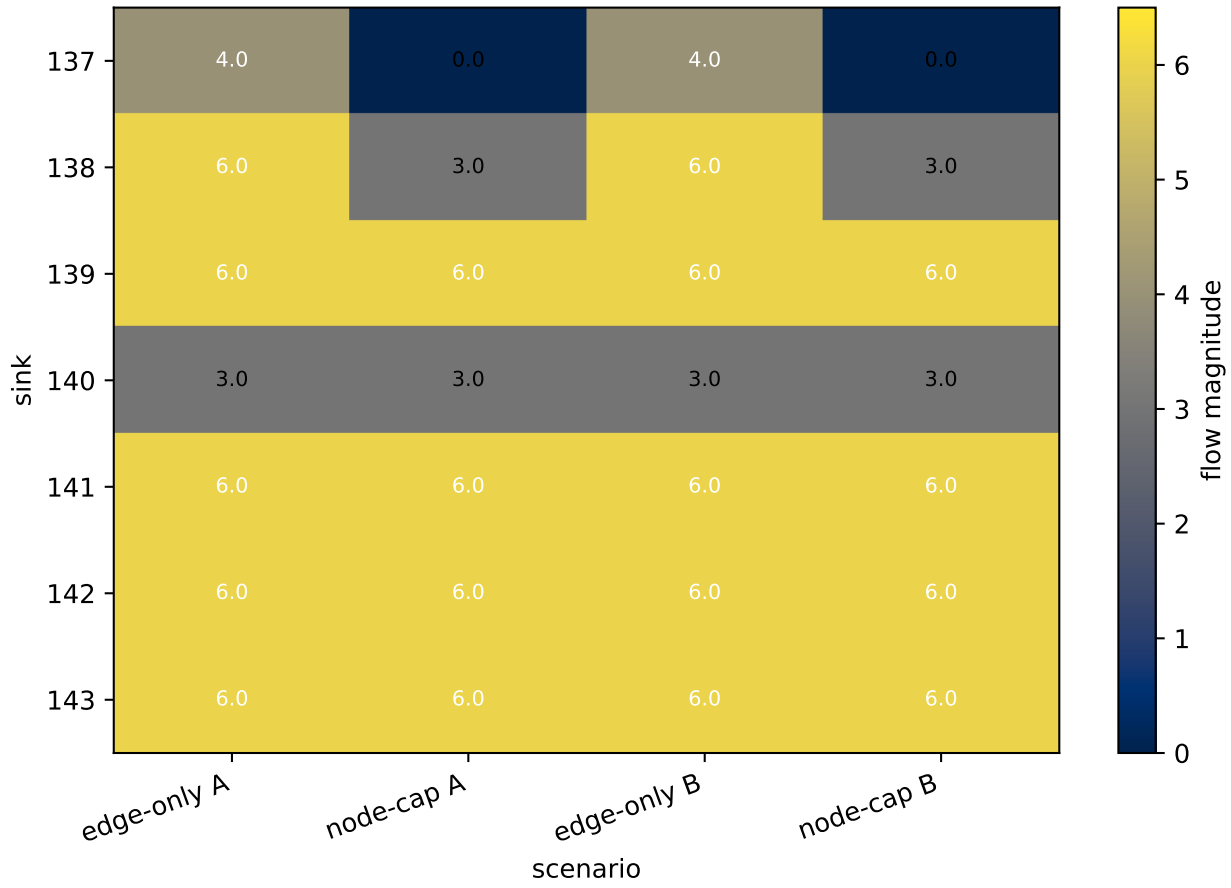


Figure 9: Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.

0.8.2 The IEEE Reliability Test System

[Numbers below are from the first run, on the structural source and sink subset. They are to be replaced by the run on the net-injection model described in the second paragraph.]

The case study is the 24-bus IEEE Reliability Test System [7, 8], taken from the MATPOWER case file `case24_ieee_rts` [9]. The system has 24 buses, 33 generating units at ten buses, 38 transmission lines of which four are double-circuit pairs, and a peak demand of 2850 MW. Line ratings in MVA are the edge capacities, and generator nameplate capacities in MW are the node capacities at the ten generator buses. The physical network is meshed, and the graph object of Chapter 3 requires a DAG, so each line was oriented by the direction of its real-power flow in a DC power flow at peak load, with generation dispatched at a uniform participation factor so that total generation equals total demand and bus 13 as the slack bus. Twenty-one of the 34 lines carry flow against the order in which the file lists their end buses. The four double-circuit pairs, which carry equal flow because their parameters are identical, become one edge each with the summed rating. No directed cycle remained after orientation, so no line was dropped. The result is a 24-node, 34-edge DAG whose zero-in-degree nodes (buses 7, 22 and 23) are exactly the generator-only buses and whose zero-out-degree nodes (buses 4, 5, 6, 8 and 19) are exactly the load-only buses, which is consistent with the orientation being physical. The orientation log and the input files are in Appendix A.

Seven buses (1, 2, 13, 15, 16, 18 and 21) carry both generation and load and are therefore neither a source nor a sink of the DAG. To measure deliverable throughput against the whole system demand, a super-source is connected to every bus whose net injection (generation minus load) is positive, with capacity equal to that net injection, and every bus whose net injection is negative is connected to a super-sink with capacity equal to its net load. Under this model the sinks' demands sum to **[D]** MW and the questions below are answered against that figure.

Under edge capacities alone the network can deliver **[2725]** MVA. With the generator capacities applied it can deliver **[1135]** MVA, so the generators are the binding constraint at peak load, which is the expected condition for a test system designed with transmission margin. The binding cut under edge capacities has **[nine]** edges, **[seven of them 175 MVA lines into the load buses, one 500 MVA line and the 1000 MVA double circuit into bus 20]** (Table 9). No bus is a single point of failure. The five most damaging single-line failures are **[the two double circuits into and out of bus 20 at 1000 MVA each, line 16–19 at 500 MVA, and lines 3–1 and 7–8 at 175 MVA each]**, and **[the most damaging generator failures are ...]**. To raise deliverable throughput by ten per cent, **[line 7–8 alone would need 447.5 MVA, and line 3–1 alone cannot reach the target at any capacity, because it is one**

of nine parallel edges in the binding cut].

Table 9: IEEE RTS-24 under the net-injection model: throughput, binding cut, most damaging failures and upgrade thresholds. **[To be completed from the rerun.]**

Quantity	Value
System demand	[–] MW
Deliverable throughput, edge capacities only	[–] MVA
Deliverable throughput, with generator capacities	[–] MVA
Binding cut (edges, capacity)	[–]
Single points of failure	[–]
Most damaging line failures (drop in MVA)	[–]
Most damaging generator failures (drop in MVA)	[–]
Upgrade to +10 per cent: required capacity on each binding line	[–]

0.8.3 Scale

The solvers were timed on four generated instances of the `genrmf` mesh family used in the DIMACS implementation challenge [10]. The generator here produces the same layered mesh topology with every edge directed forward, so that the instances are DAGs, and they are therefore a variant of the published family and not its output. Sizes are 360 nodes and 924 edges, 3,332 and 9,324, 27,040 and 78,364, and 36,992 and 107,644, with capacities up to 1000 from a recorded seed. Table 10 gives the warm second-call times. Dinic is the fastest of the three on every instance and completes the largest in under a second; push-relabel is within a factor of four; Edmonds-Karp was not run above 50,000 edges, where its $O(VE^2)$ bound makes it impractical. The external oracle, which agrees with all three on every instance, takes 29 s on the largest. The comparison is between implementations on the same machine and says nothing about the algorithms in general; it establishes that the toolkit's default solver handles networks two orders of magnitude larger than any in the corpus in interactive time.

Table 10: Warm second-call solve times on DAG variants of the `genrmf` mesh family. Edmonds-Karp was not run above 50,000 edges.

Instance	Nodes	Edges	Dinic (s)	Push-relabel (s)	Edmonds-Karp (s)
small	360	924	0.002	0.097	0.007
medium	3,332	9,324	0.017	0.019	0.179
large	27,040	78,364	0.961	1.520	–
xlarge	36,992	107,644	0.657	2.277	–

0.9 Discussion

The toolkit is point-valued. An interval or probability-box capacity is not propagated, and running the best-case and worst-case capacities gives two solves whose relation to the true range of the maximum flow is a bound only because the maximum flow is monotone in every capacity. Extending the toolkit to intervals by that monotonicity argument is direct for the flow value; the cut structure, the failure rankings and the thresholds are not monotone and would need the same kind of treatment that the floats received in Chapter 7.

Exactness in this chapter is on the evaluated candidates. The sensitivity and failure modules restrict their candidates to the saturated edges and to the edges of some minimum cut, and their results are exact on those sets. An edge outside the candidate set cannot change the maximum flow by a small change in its capacity, but a large enough change can, and the k-edge failure analysis is exhaustive only over the candidate set.

The toolkit computes no probability. It does not answer the stochastic-flow question of Chapter 2, the probability that a stated demand can be met, and it does not weight a failure by its likelihood. That combination is a natural extension, since the probability toolkit of Chapter 5 gives the probability that a component operates, but it is not built.

All analyses are on a single fixed-capacity instance. Time-varying degradation and adaptive rerouting are outside the model, and the degradation scenarios of Section 0.5.4 are a sequence of separate instances.

The global edge and node connectivity of a DAG, defined as the minimum over all ordered node pairs of the unit-capacity maximum flow, is zero, because a sink has no outgoing edge and every DAG has a sink. The implementation computed exactly that definition and therefore returned zero on every network, which the validation confirmed independently. The definition that answers the redundancy question is the minimum over declared source and sink pairs, and it is listed as a change in Chapter 11; the disjoint-path counts of Section 0.5.2, which are per pair, are unaffected.

Two further extensions follow from the model. Minimum-cost flow, in which each edge carries a cost per unit as well as a capacity, would answer how a given demand is met most cheaply, and a time-expanded network would carry the flow model across a planning horizon. Both are listed in Chapter 11.

0.10 Summary

In this chapter the Capacity Flow Toolkit has been presented. The flow model has been stated for a capacitated DAG, with the reductions for several sources and sinks and for node capacities proved exact, and three maximum-flow solvers have been provided on the graph object of Chapter 3, each result checked for feasibility, conservation and

agreement between flow and cut. Six questions have been answered by modules that share one baseline solve: throughput and margin, the binding cut and the lattice of every equivalent cut, the most damaging failures and degradations, the marginal value and range of each edge, degradation and upgrade thresholds by a partition-change search, and redundancy and single points of failure. The toolkit has been validated against an external oracle and brute-force enumeration on twelve networks, timed on generated instances of up to 10^5 edges, and applied to the IEEE Reliability Test System, on which the generator capacities and not the lines bind at peak load. The toolkit is point-valued, and its extension to interval capacities and to probability-weighted measures is left to future work.

Bibliography

[1] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: 10.4153/CJM-1956-045-5

[2] L. R. Ford and D. R. Fulkerson, *Flows in Networks*. Princeton, NJ: Princeton University Press, 1962.

[3] Z. W. Birnbaum, "On the importance of different components in a multicomponent system," in *Multivariate Analysis II*, P. R. Krishnaiah, Ed., New York: Academic Press, 1969, pp. 581–592.

[4] D. Bertsimas, E. Nasrabadi and S. Stiller, "Robust and adaptive network flows," *Operations Research*, vol. 61, no. 5, pp. 1218–1242, 2013. DOI: 10.1287/opre.2013.1200

[5] P. Doulliez and E. Jamoulle, "Transportation networks with random arc capacities," *Revue française d'automatique, informatique, recherche opérationnelle. Recherche opérationnelle*, vol. 6, no. V3, pp. 45–59, 1972. [Online]. Available: https://www.numdam.org/item/RO_1972__6_3_45_0/

[6] JuliaGraphs contributors, *GraphsFlows.jl: Flow algorithms on top of Graphs.jl*, https://github.com/JuliaGraphs/GraphsFlows.jl, 2024.

[7] Reliability Test System Task Force of the Application of Probability Methods Subcommittee, "IEEE reliability test system," *IEEE Transactions on Power Apparatus and Systems*, vol. PAS-98, no. 6, pp. 2047–2054, 1979. DOI: 10.1109/TPAS.1979.319398

[8] C. Grigg et al., "The IEEE reliability test system-1996. a report prepared by the reliability test system task force of the application of probability methods subcommittee," *IEEE Transactions on Power Systems*, vol. 14, no. 3, pp. 1010–1020, 1999. DOI: 10.1109/59.780914

[9] R. D. Zimmerman, C. E. Murillo-Sánchez and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, 2011. DOI: 10.1109/TPWRS.2010.2051168

[10] D. S. Johnson and C. C. McGeoch, Eds., *Network Flows and Matching: First DIMACS Implementation Challenge* (DIMACS Series in Discrete Mathematics and Theoretical Computer Science). American Mathematical Society, 1993, vol. 12.